2026

# Objectives, Project Plan and Requirements

3:2 ANALYTICS

EXPENSE MANAGEMENT APP

# EXPENSE MANAGEMENT APP

## GENERAL OBJETIVES

Develop a mobile application that helps teenagers (ages 13–18) understand and practice basic financial skills — including budgeting, saving, and responsible spending — through interactive and gamified learning tools. The app aims to bridge the gap in financial education by providing an accessible, engaging, and age-appropriate platform where teenagers can learn, practice, and apply real-world money management skills at their own pace.

## SPECIFIC OBJETIVES

**1. Improve Financial Knowledge** Teach foundational financial concepts such as budgeting, saving, compound interest, and responsible spending. Content will be broken into digestible lessons tailored to a teenager's level of understanding, avoiding overly technical language while still introducing meaningful financial vocabulary and principles.

**2. Encourage Good Money Habits** Help teenagers develop consistent money habits by allowing them to log and track their spending, set realistic savings goals, and reflect on their financial choices over time. The app will reinforce positive behavior through streaks, reminders, and progress feedback.

**3. Make Learning Engaging** Leverage gamification techniques — including quizzes, daily challenges, leaderboards, and reward systems — to keep users motivated and returning to the app regularly. Learning should feel less like a classroom and more like an interactive experience teenagers actually want to participate in.

**4. Promote Financial Decision-Making** Place users in simulated real-life financial scenarios where they must make spending, saving, or investment decisions and see the consequences play out. These simulations help build critical thinking and reinforce the idea that financial choices have real-world outcomes.

**5. Provide Progress Tracking** Give users a clear view of how far they've come. Through dashboards, completed modules, earned badges, and quiz history, teenagers can see their growth over time — building confidence and motivating continued engagement.

## MEASUREMENT INDICATORS

The following KPIs are used to evaluate whether the app is successfully meeting its objectives. These should be tracked through in-app analytics and user data collected during testing and post-launch.

| Objective | Indicator | Target |
|---|---|---|
| **Increase financial knowledge** | Quiz score improvement from baseline to post-use | +30% improvement after consistent app use |
| **Encourage saving habits** | Number of savings goals created per user | ≥ 2 savings goals per active user |
| **Maintain engagement** | Weekly active user retention rate | ≥ 60% of users active each week |
| **Improve financial decisions** | Completion rate of simulation challenges | ≥ 70% of users complete assigned simulations |

| | | |
|---|---|---|
| **Learning progress** | Number of modules completed per user | ≥ 80% of users complete at least 3 modules |

These targets are realistic benchmarks for an early-stage app. They should be revisited after the first round of user testing and adjusted based on real data collected during the testing phase in Week 8.

## REQUIREMENTS DEFINITION

### FUNCTIONAL REQUIREMENTS

These are the core features the app must include to fulfill its objectives. Every feature listed below directly supports at least one of the specific objectives defined in Section 2.

- **User Registration and Profile** – Users create an account with basic information (name, age range, optional avatar) to personalize their experience and track individual progress over time.
- **Budget Tracker** – An interactive tool that allows users to log income (e.g., allowance, part-time job) and expenses, and visualize where their money is going through simple charts or breakdowns.
- **Savings Goal Planner** – Users can set specific savings goals (e.g., "Save $50 for new headphones"), define a target date, and track progress toward that goal with visual indicators.
- **Interactive Financial Lessons** – Short, structured lessons covering key financial topics such as budgeting basics, the importance of saving, how interest works, and the difference between needs and wants.
- **Quizzes and Mini-Games** – Knowledge checks and interactive games that reinforce lesson content. These should be short, fun, and rewarding to complete, with scores tracked over time to show improvement.
- **Financial Simulations** – Scenario-based challenges where users make financial decisions (e.g., "You have $200 — how do you split it between rent, food, and savings?") and receive feedback on the outcomes of their choices.
- **Progress Dashboard** – A personalized home screen showing completed lessons, quiz scores, savings goal progress, earned badges, and overall app activity at a glance.
- **Achievement Badges and Rewards** – A reward system that recognizes milestones such as completing a module, reaching a savings goal, or logging in for consecutive days, encouraging continued use.
- **Notifications and Reminders** – Optional push notifications to remind users to check in, continue a lesson, or review their budget — helping build consistent financial habits.

### NON-FUNCTIONAL REQUIREMENTS

These requirements define the quality standards the app must meet, independent of specific features.

- **Usability** – The interface must be clean, intuitive, and designed specifically for teenagers. Navigation should be straightforward, with minimal steps required to access any core feature. Avoid cluttered layouts or overly complex menus.

- **Accessibility** – Text should be legible at standard screen sizes, with sufficient contrast and font sizes. Where possible, the app should support basic accessibility features such as screen reader compatibility.
- **Performance** – The app should load within 3 seconds under normal network conditions. Transitions between screens should be smooth, and no core feature should experience lag or delay during typical use.
- **Security** – All user data must be stored securely and never shared with third parties without consent. Since the app targets users under 18, the development team must account for **COPPA** (Children's Online Privacy Protection Act) compliance in the US, or equivalent data protection regulations depending on the target region (e.g., GDPR-K in Europe).
- **Compatibility** – The app must function on both **Android and iOS** platforms. The team should decide early in the development phase whether to use a **cross-platform framework such as React Native or Flutter**, or build a responsive web app, as this decision will significantly impact the development timeline and resource requirements.
- **Scalability** – While the initial version is a demo/prototype, the architecture should allow for additional lessons, features, or user accounts to be added in future iterations without requiring a full rebuild.

## MILESTONES DEFINITION

The project is structured across a 9-week timeline. Each milestone represents a critical checkpoint that must be completed before the next phase begins. The team should confirm the actual calendar start date and map these weeks to specific dates to ensure accountability.

| Milestone | Description | Time |
|---|---|---|
| **Project Planning** | Define objectives, scope, key features, and assign team roles and responsibilities | Week 1 |
| **Requirements Finalized** | All functional and non-functional requirements reviewed, discussed, and approved by the full team | Week 2 |
| **Prototype Design** | Wireframes created for all core screens; UX flow mapped out and reviewed for usability | Week 3 |
| **Development Phase** | Core features built and integrated, including tracker, lessons, quizzes, simulations, and dashboard | Weeks 4–7 |
| **Testing** | Internal bug fixing, usability testing with sample users, and feedback-based revisions | Week 8 |
| **Final Version** | Fully functional app demo ready for presentation, with all features working and content complete | Week 9 |

**Important note for the team:** These are relative weeks. Before development begins, agree on an actual start date and convert these into calendar deadlines. Each team member should know which milestone their deliverables fall under and what is expected of them at each checkpoint.

## ALIGNMENT WITH RUBRIC

This section maps the project directly to the evaluation rubric to ensure the team is building toward the right outcomes from the start.

This section maps the project directly to the evaluation rubric to ensure the team is building toward the right outcomes from the start.

| Rubric Criterion | How the Project Meets It |
|---|---|
| **Problem Definition** | The app directly addresses a well-documented gap — most teenagers receive little to no formal financial education, leaving them unprepared to manage money as adults. The problem is specific, relevant, and backed by a clear target audience. |
| **Innovation** | The app combines gamification, real-life simulations, and personalized goal tracking to make financial education interactive rather than passive. This goes beyond textbook learning and creates hands-on experience. |
| **Feasibility** | All features listed are achievable within the 9-week timeline using existing mobile development tools and frameworks. The scope is realistic for a small team working on a demo or prototype. |
| **Impact** | By building financial knowledge and habits during the teenage years, the app has the potential to create lasting positive change in how users manage money throughout their adult lives. |
| **Organization** | The project is clearly structured with defined objectives, measurable KPIs, detailed requirements, and a phased milestone plan — giving every team member a clear roadmap to follow. |

.

## WORK PACKAGES AND TASKS

The development of the Financial Literacy App will be organized into several **work packages (WPs)** that structure the project lifecycle from planning to final delivery. Each work package contains specific tasks, required inputs, expected outputs, and responsible team members.

## WP1 – PROJECT PLANNING

Define the scope of the application, clarify the objectives, and distribute responsibilities among team members before starting development.

| Task | Description | Start | End | Inputs | Outputs | People |
|---|---|---|---|---|---|---|
| **Project scope definition** | Define the limits of the prototype and main features | Week 1 | Week 1 | Project idea | Defined project scope | All team members |
| **Role allocation** | Assign responsibilities (Person 1–4) | Week 1 | Week 1 | Team discussion | Defined responsibilities | All |
| **Planning document** | Draft project structure and milestones | Week 1 | Week 1 | Objectives and requirements | Project plan draft | Person 2 |

## WP2 – REQUIREMENTS AND SYSTEM DESIGN

Translate project objectives into clear functional and technical requirements, ensuring the system

| Task | Description | Start | End | Inputs | Outputs | People |
|---|---|---|---|---|---|---|
| **Functional requirements review** | Validate app features (budget tracker, lessons, simulations) | Week 2 | Week 2 | Requirements definition | Approved feature list | Person 1 & 2 |
| **Non-functional requirements definition** | Define usability, performance and security constraints | Week 2 | Week 2 | System needs | Quality standards | Person 2 |
| **UX flow planning** | Define navigation between screens | Week 2 | Week 3 | Feature list | User flow map | Person 3 |

## WP3 – PROTOTYPE DESIGN

Design the visual structure of the application and the user interface before development begins.

| Task | Description | Start | End | Inputs | Outputs | People |
|---|---|---|---|---|---|---|
| **Wireframe design** | Create basic layouts of key screens | Week 3 | Week 3 | UX flow | Wireframes | Person 3 |
| **Interface validation** | Review usability and navigation clarity | Week 3 | Week 3 | Wireframes | Approved UI structure | Team |

## WP4 – APPLICATION DEVELOPMENT

Develop the core features of the mobile application and integrate the main modules of the system.

| Task | Description | Start | End | Inputs | Outputs | People |
|---|---|---|---|---|---|---|
| **User account system** | Implement registration and profile system | Week 4 | Week 5 | Requirements | Working login system | Person 2 |
| **Budget tracker implementation** | Develop income/expense logging tool | Week 4 | Week 6 | Functional specs | Budget tracker module | Team |
| **Financial lessons module** | Add structured financial learning content | Week 5 | Week 6 | Lesson content | Learning module | Person 4 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **Gamification features** | Implement quizzes, challenges and badges | Week 6 | Week 7 | Learning content | Gamification system | Team |
| **Dashboard integration** | Combine progress tracking features | Week 6 | Week 7 | All modules | Functional dashboard | Team |

## WP5 – TESTING AND VALIDATION

Ensure the application functions correctly, fix bugs, and validate usability with sample users.

| Task | Description | Start | End | Inputs | Outputs | People |
|---|---|---|---|---|---|---|
| **Functional testing** | Verify that all features work correctly | Week 8 | Week 8 | App prototype | Bug report | Team |
| **Usability testing** | Collect feedback from test users | Week 8 | Week 8 | Prototype | Usability improvements | Team |
| **Bug fixing** | Resolve detected issues | Week 8 | Week 8 | Bug report | Stable application | Team |

## WP6 – FINAL DELIVERY

Prepare the final version of the application prototype and the presentation materials.

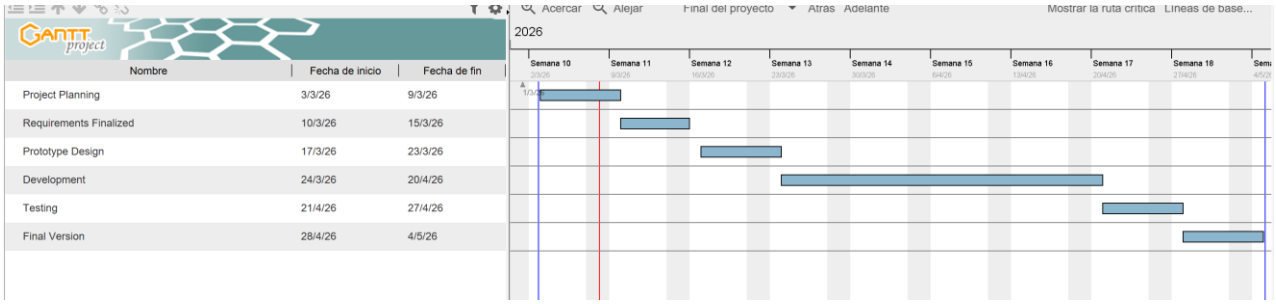
| Task | Description | Start | End | Inputs | Outputs | People |
|---|---|---|---|---|---|---|
| **Final system integration** | Ensure all features work together | Week 9 | Week 9 | Tested modules | Complete app demo | Team |
| **Documentation and presentation** | Prepare final report and demo | Week 9 | Week 9 | Final prototype | Final presentation | Team |

## RESOURCE ALLOCATION

Each team member contributes according to their assigned role:

| Team Member | Main Responsibilities |
|---|---|
| **Person 1** | Planning and project organization |
| **Person 2** | Defining objectives and system requirements |
| **Person 3** | Use case diagram and user experience design |
| **Person 4** | Design system structure and functionality |
| **Person 5** | Reviewing the prototype and measuring results |

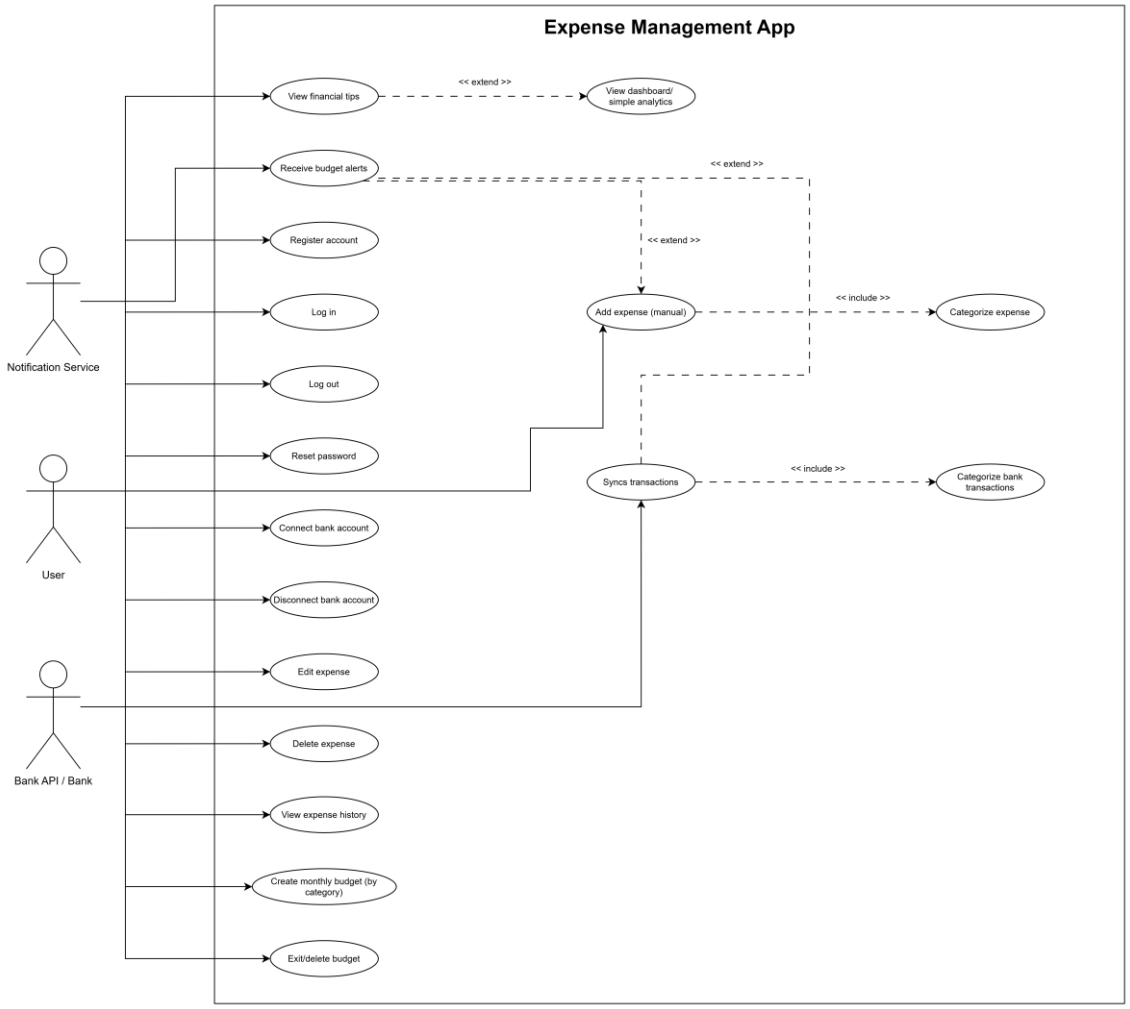
# GANTT DIAGRAM



| Nombre | Fecha de inicio | Fecha de fin |
|---|---|---|
| Project Planning | 3/3/26 | 9/3/26 |
| Requirements Finalized | 10/3/26 | 15/3/26 |
| Prototype Design | 17/3/26 | 23/3/26 |
| Development | 24/3/26 | 20/4/26 |
| Testing | 21/4/26 | 27/4/26 |
| Final Version | 28/4/26 | 4/5/26 |



# USE CASE DIAGRAM



**Detailed Use Case Descriptions**

From all the use cases shown in the diagram, we picked the ones that actually matter the most to understand how the app works end to end. We didn't include stuff like Log out or Reset password because honestly, those are pretty self-explanatory and don't add much value describing them in detail.

The selected use cases are:

1. Register account
2. Add expense (manual)
3. Create monthly budget (by category)
4. Connect bank account
5. Sync transactions
6. View expense history
7. View dashboard / simple analytics

## UC-01: REGISTER ACCOUNT

| Field | Description |
|---|---|
| **Name** | Register account |
| **Short description** | A new user creates an account by entering their basic information so they can start using the app. |
| **Precondition** | The user does not have an existing account. The app is installed and there is internet connection. |
| **Postcondition** | The user account is created and stored in the system. The user is redirected to the main screen. |
| **Error situations** | The email entered is already registered. Required fields are left empty. Password doesn't meet minimum requirements. |
| **System state on error** | No account is created. The user stays on the registration screen with an error message shown. |
| **Actors** | User |
| **Trigger** | The user opens the app for the first time and taps "Create account". |
| **Standard process** | (1) User opens the app and selects "Register". (2) User enters name, email, and password. (3) System validates that the email is not already in use. (4) System validates that all fields are filled and the password meets the requirements. (5) System creates the account and stores the data. (6) System confirms registration and takes the user to the home screen. |
| **Alternative processes** | (3') The email is already registered → System shows an error and asks the user to log in or use a different email. (4') Password is too weak → System asks the user to enter a stronger password. |

## UC-02: ADD EXPENSE (MANUAL)

| Field | Description |
|---|---|
| **Name** | Add expense (manual) |
| **Short description** | The user manually logs an expense by entering the amount, description, and date. The system automatically categorizes it. |
| **Precondition** | The user is logged in. |
| **Postcondition** | The expense is saved in the system and assigned to a category. The user's balance and history are updated. |
| **Error situations** | The user leaves the amount field empty. The user enters an invalid amount (negative number, letters, etc.). |
| **System state on error** | The expense is not saved. The user stays on the expense form with an error message. |
| **Actors** | User |
| **Trigger** | The user taps the "Add expense" button from the home screen or the expense section. |
| **Standard process** | (1) User selects "Add expense". (2) User enters amount, description, and date. (3) System validates the data. (4) System automatically categorizes the expense (<<include>> Categorize expense). (5) System saves the expense. (6) System shows confirmation and updates the expense list. |
| **Alternative processes** | (4') The user disagrees with the automatic category → User manually selects a different category before saving. (2') The user wants to add an expense without a description → System saves it with "No description" as default. |

## UC-03: CREATE MONTHLY BUDGET (BY CATEGORY)

| Field | Description |
|---|---|
| **Name** | Create monthly budget (by category) |
| **Short description** | The user sets a spending limit for one or more categories for the current month. |
| **Precondition** | The user is logged in. There is no existing budget for the selected category in the current month. |
| **Postcondition** | The budget is created and stored. The system will now track spending against this limit. |
| **Error situations** | The user enters an invalid amount (zero, negative). The user tries to create a budget for a category that already has one this month. |
| **System state on error** | No budget is created. The user stays on the budget creation screen with the corresponding error message. |
| **Actors** | User |
| **Trigger** | The user navigates to the budget section and selects "Create budget". |

| | |
|---|---|
| **Standard process** | (1) User opens the budget section. (2) User selects a category (food, transport, entertainment, etc.). (3) User enters the monthly spending limit. (4) System validates that no budget exists for that category this month. (5) System saves the budget. (6) System shows the budget in the overview with a progress bar at 0%. |
| **Alternative processes** | (4') A budget already exists for that category → System asks if the user wants to edit the existing one instead. (3') User wants to set budgets for multiple categories at once → System allows adding more categories before saving. |

## UC-04: CONNECT BANK ACCOUNT

| Field | Description |
|---|---|
| **Name** | Connect bank account |
| **Short description** | The user links their bank account to the app so that transactions can be synced automatically. |
| **Precondition** | The user is logged in. The user does not already have the same bank account connected. |
| **Postcondition** | The bank account is linked to the user's profile. The system is ready to sync transactions from this account. |
| **Error situations** | The bank's API is unavailable. The user enters wrong banking credentials. Authentication fails. |
| **System state on error** | The bank account is not connected. The user is informed about the error and can retry. |
| **Actors** | User, Bank API / Bank |
| **Trigger** | The user goes to settings or the bank section and taps "Connect bank account". |
| **Standard process** | (1) User selects "Connect bank account". (2) System shows a list of supported banks. (3) User selects their bank. (4) System redirects to the bank's authentication page. (5) User enters their banking credentials. (6) Bank API validates the credentials and authorizes the connection. (7) System confirms the connection and stores the link. (8) System shows the bank account in the user's profile. |
| **Alternative processes** | (6') Credentials are wrong → Bank API rejects the login → System shows an error and lets the user try again. (6'') Bank API is down → System informs the user to try again later. |

## UC-05: SYNC TRANSACTIONS

| Field | Description |
|---|---|
| **Name** | Sync transactions |
| **Short description** | The system retrieves the latest transactions from the user's connected bank account and categorizes them automatically. |
| **Precondition** | The user is logged in. At least one bank account is connected. |
| **Postcondition** | New transactions from the bank are imported, categorized, and visible in the expense history. |
| **Error situations** | The bank API is unavailable. The connection token has expired and needs re-authentication. |

| | |
|---|---|
| **System state on error** | No new transactions are imported. The user is notified about the sync failure. |
| **Actors** | User, Bank API / Bank |
| **Trigger** | The user taps "Sync now", or the system performs an automatic periodic sync. |
| **Standard process** | (1) System sends a request to the Bank API for new transactions. (2) Bank API returns the list of recent transactions. (3) System processes each transaction and categorizes it automatically (<<include>> Categorize bank transactions). (4) System stores the transactions. (5) System updates the expense history and notifies the user of how many new transactions were imported. |
| **Alternative processes** | (1') The sync is triggered automatically in the background → Steps 2–5 happen the same way, but no visual feedback is shown unless there are errors. (3') A transaction can't be categorized automatically → System assigns it to "Uncategorized" and the user can fix it later. |

## UC-06: VIEW EXPENSE HISTORY

| Field | Description |
|---|---|
| **Name** | View expense history |
| **Short description** | The user checks their past expenses, with the ability to filter by date, category, or source (manual vs bank). |
| **Precondition** | The user is logged in. |
| **Postcondition** | The system state doesn't change. The user sees a list of their recorded expenses. |
| **Error situations** | No expenses found for the selected filters. |
| **System state on error** | System remains unchanged. An empty state message is shown ("No expenses found"). |
| **Actors** | User |
| **Trigger** | The user navigates to the expense history section. |
| **Standard process** | (1) User opens the expense history screen. (2) System loads and displays all expenses, sorted by most recent first. (3) User scrolls through the list to review their expenses. |
| **Alternative processes** | (2') User applies a filter (by date range, category, or source) → System shows only the expenses matching the filter. (2'') There are no expenses at all → System shows an empty state with a suggestion to add one. |

## UC-07: VIEW DASHBOARD / SIMPLE ANALYTICS

| Field | Description |
|---|---|
| **Name** | View dashboard / simple analytics |
| **Short description** | The user accesses a summary of their financial activity including spending by category, budget progress, and recent transactions. |
| **Precondition** | The user is logged in. |
| **Postcondition** | No data is changed. The user sees their financial overview. |

| | |
|---|---|
| **Error situations** | There's not enough data to generate meaningful charts (e.g., brand new account with no expenses). |
| **System state on error** | System remains unchanged. Dashboard shows placeholder content or tips to get started. |
| **Actors** | User |
| **Trigger** | The user opens the app (if dashboard is the home screen) or taps on "Dashboard" in the navigation. |
| **Standard process** | (1) User opens the dashboard. (2) System loads spending data, budget progress, and recent transactions. (3) System displays charts showing spending by category, budget usage percentage, and a short list of the latest expenses. (4) User reviews the information. |
| **Alternative processes** | (2') The user has connected a bank account → Dashboard also shows synced transaction summaries alongside manual expenses. (3') The user taps on a specific category chart → System navigates to the filtered expense history for that category (this is the <<extend>> relationship from View financial tips). |

## NOTE ON SCOPE

All the use cases described here are realistic and doable within the development phase (Weeks 4–7) of the project timeline. We intentionally kept the flows simple and focused on what a working prototype would need. More complex stuff like multi-currency support, recurring expenses, or exporting reports could be added in future versions, but for now this covers the core functionality of the app.